

codata

Table of Contents

codata	1
codata_cli	3
codata_version	4
codata_constant_type	5

NAME

codata - library for fundamental physical constants

LIBRARY

codata (**-libcodata**, **-lcodata**)

SYNOPSIS

Fortran: *use codata*

C: *include "codata.h"*

Python: *import pycodata*

DESCRIPTION

codata(3) is a Fortran library providing the fundamental physical constants published by the CODATA.

Fortran API

exposes the constants as derived type `codata_constant_type` which carries:

- name
- value
- uncertainty
- unit

C API exposes a structure `codata_constant_type` that defines the same members as in Fortran:

- name
- value
- uncertainty
- unit

Python exposes a dictionary with the keys being:

- the name
- the value
- the uncertainty
- the unit

NOTES

The latest values (2022) do not have the year as a suffix in their name whereas older values (2010, 2014, 2018) feature the year as a suffix. All constants are declared with the same variable name in Fortran, C, and Python.

EXAMPLE

Example in Fortran

```
program example_in_f
  use codata
  implicit none
  print '(A)', '##### EXAMPLE IN FORTRAN #####'
  print '(A)', '# VERSION'
  print *, "version = ", version()
  print '(A)', '# CONSTANTS'
  print *, "c = ", SPEED_OF_LIGHT_IN_VACUUM%value
  print '(A)', '# UNCERTAINTY'
  print *, "u(c) = ", SPEED_OF_LIGHT_IN_VACUUM%uncertainty
  print '(A)', '# OLDER VALUES'
```

```

print '(A, F23.16)', "Mu_2022(latest) = ", MOLAR_MASS_CONSTANT%value
print '(A, F23.16)', "Mu_2018 = ", MOLAR_MASS_CONSTANT_2018%value
print '(A, F23.16)', "Mu_2014 = ", MOLAR_MASS_CONSTANT_2014%value
print '(A, F23.16)', "Mu_2010 = ", MOLAR_MASS_CONSTANT_2010%value
end program

```

Example in C

```

#include <stdio.h>
#include "codata.h"

int main(void){
    printf("%f", c);
    printf("##### EXAMPLE IN C #####0);
    printf("%s0, "# VERSION");
    printf("version = %s0, codata_version());
    printf("%s0, "# CONSTANTS");
    printf("c = %f0, SPEED_OF_LIGHT_IN_VACUUM.value);
    printf("%s0, "# UNCERTAINTY");
    printf("u(c) = %f0, SPEED_OF_LIGHT_IN_VACUUM.uncertainty);
    printf("%s0, "# OLDER VALUES");
    printf("Mu_2022(latest) = %23.16f0, MOLAR_MASS_CONSTANT.value);
    printf("Mu_2018 = %23.16f0, MOLAR_MASS_CONSTANT_2018.value);
    printf("Mu_2014 = %23.16f0, MOLAR_MASS_CONSTANT_2014.value);
    printf("Mu_2010 = %23.16f0, MOLAR_MASS_CONSTANT_2010.value);
    return 0;
}

```

Example in Python

```

import sys
sys.path.insert(0, "../py/src/")
import pycodata
print("##### EXAMPLE IN PYTHON #####")
print("# VERSION")
print(f"version = {pycodata.__version__}")
print("# Constants")
print("c =", pycodata.SPEED_OF_LIGHT_IN_VACUUM["value"])
print("# UNCERTAINTY")
print("u(c) = ", pycodata.SPEED_OF_LIGHT_IN_VACUUM["uncertainty"])
print("# OLDER VALUES")
print("Mu_2022 = ", pycodata.MOLAR_MASS_CONSTANT["value"])
print("Mu_2018 = ", pycodata.MOLAR_MASS_CONSTANT_2018["value"])
print("Mu_2014 = ", pycodata.MOLAR_MASS_CONSTANT_2014["value"])
print("Mu_2010 = ", pycodata.MOLAR_MASS_CONSTANT_2010["value"])

```

SEE ALSO

codata(1), codata_version(3), codata_constant_type(3)

NAME

codata - print fundamental physical constants

SYNOPSIS

codata [OPTION...] [REGEX_PATTERN...]

DESCRIPTION

codata(1) is a command line interface which prints all the **codata** constants.

The current values are from 2022. Older values can be retrieved if needed and the output can be filtered with REGEX PATTERNS.

OPTIONS

--year, -y YEAR

CODATA constants: 2022, 2018, 2014, 2010

--pattern, -p PATTERN

Regex pattern for filtering the constants.

--value, -a

Show only the value.

--error, -e

Show only the uncertainty.

--usage Show usage text and exit.

--version, -v

Show version information and exit.

--help, -h

Show help text and exit.

NOTES

You may replace the default options from a file if your first options begin with @file. Initial options will then be read from the "response file" "file.rsp" in the current directory.

If "file" does not exist or cannot be read, then an error occurs and the program stops. Each line of the file is prefixed with "options" and interpreted as a separate argument. The file itself may not contain @file arguments. That is, it is not processed recursively.

For more information on response files see https://urbanjost.github.io/M_CLI2/set_args.3m_cli2.html

EXAMPLE

Minimal example

```
codata
codata -y 2018 molar electron
codata -y 2014 -p molar.*gas,electron.*eV
codata [B,b]oltzmann.*eV
```

SEE ALSO

codata(3)

NAME

version - get the version of the Fortran library codata

LIBRARY

codata (**-libcodata**, **-lcodata**)

SYNOPSIS

Fortran: **character**(*len=:*), pointer **version**()

C: char* **codata_version**()

Python: pycodata.__version__

DESCRIPTION

Get the library version.

EXAMPLE

Fortran:

```
use codata
print *, "version = ", version()
```

C:

```
include <stdio.h>
include "codata.h"
printf("version = %s0, codata_version());
```

Python:

```
import pycodata
print(f"version = {pycodata.__version__}")
```

SEE ALSO

codata(3)

NAME

codata_constant_type - data type defining a constant

SYNOPSIS

Fortran: **type**(*codata_constant_type*) :: c

C: struct *codata_constant_type* c

DESCRIPTION

Defines a constant with the name, the value, the uncertainty and the unit.

EXAMPLE

Fortran:

```
use codata
type(codata_constant_type), pointer :: c
c = SPEED_OF_LIGHT_IN_VACUUM
```

C:

```
struct codata_constant_type c;
c = SPEED_OF_LIGHT_IN_VACUUM;
```

SEE ALSO

codata(3)